DL lec 11

Generative adversarial networks (GANs)

Dr. Mohammad Mhawish

Generative adversarial networks (GANs)

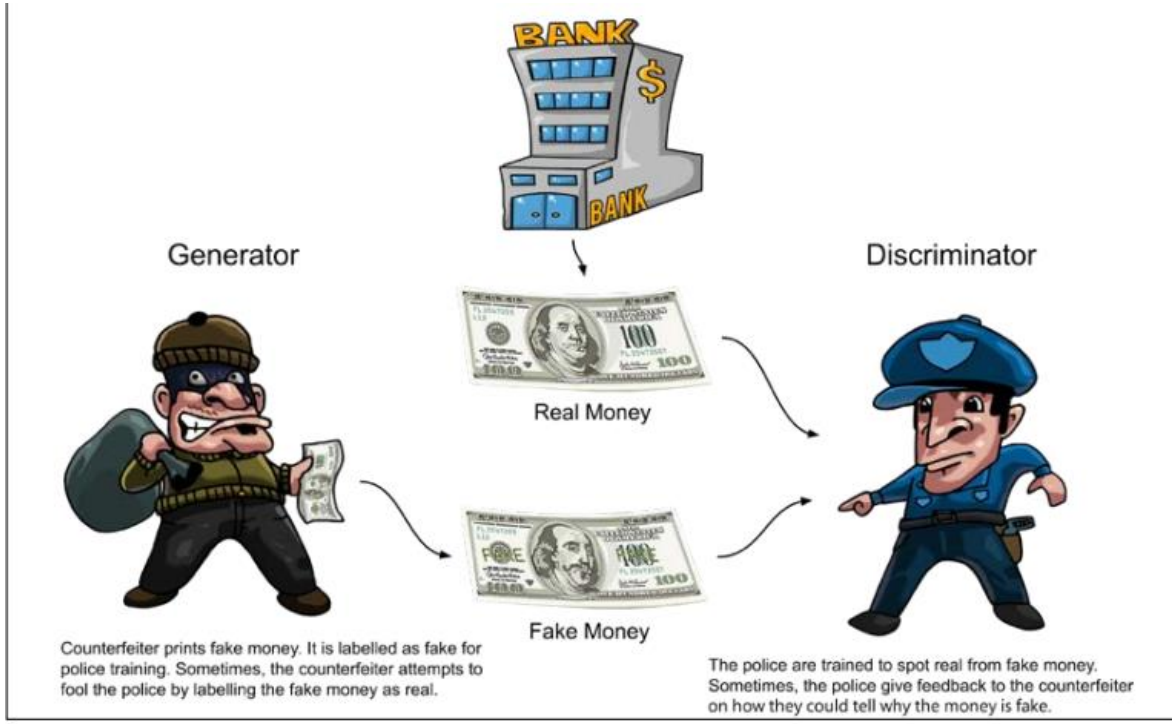
Dr. Mohammad Mhawish

- Generative adversarial networks (GANs) are a new type of neural architecture introduced by Ian Goodfellow and other researchers at the University of Montreal, including Yoshua Bengio, in 2014 .

GAN architecture

GANs are based on the idea of *adversarial training*. The GAN architecture basically consists of two neural networks that compete against each other:

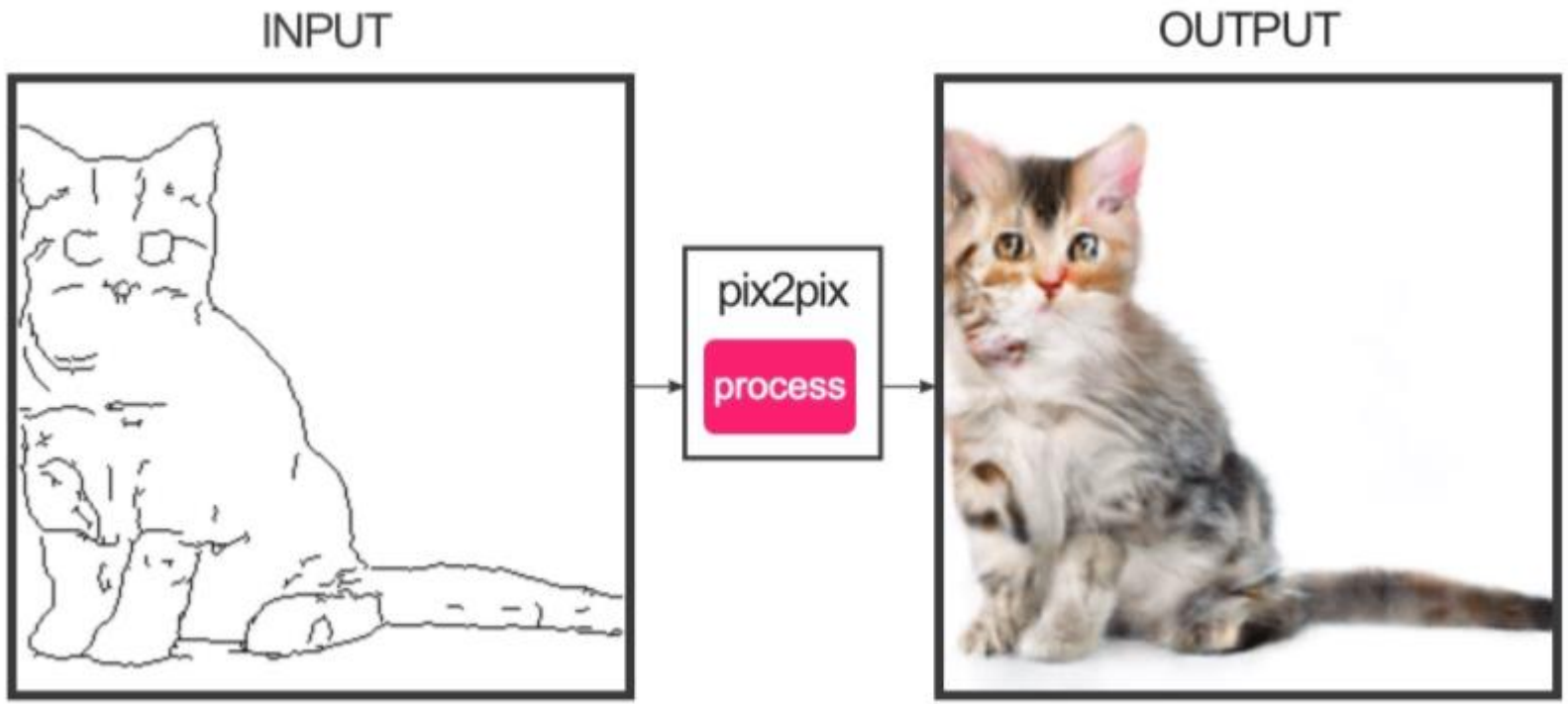
- The *generator* tries to convert random noise into observations that look as if they have been sampled from the original dataset.
- The *discriminator* tries to predict whether an observation comes from the original dataset or is one of the generator's forgeries.



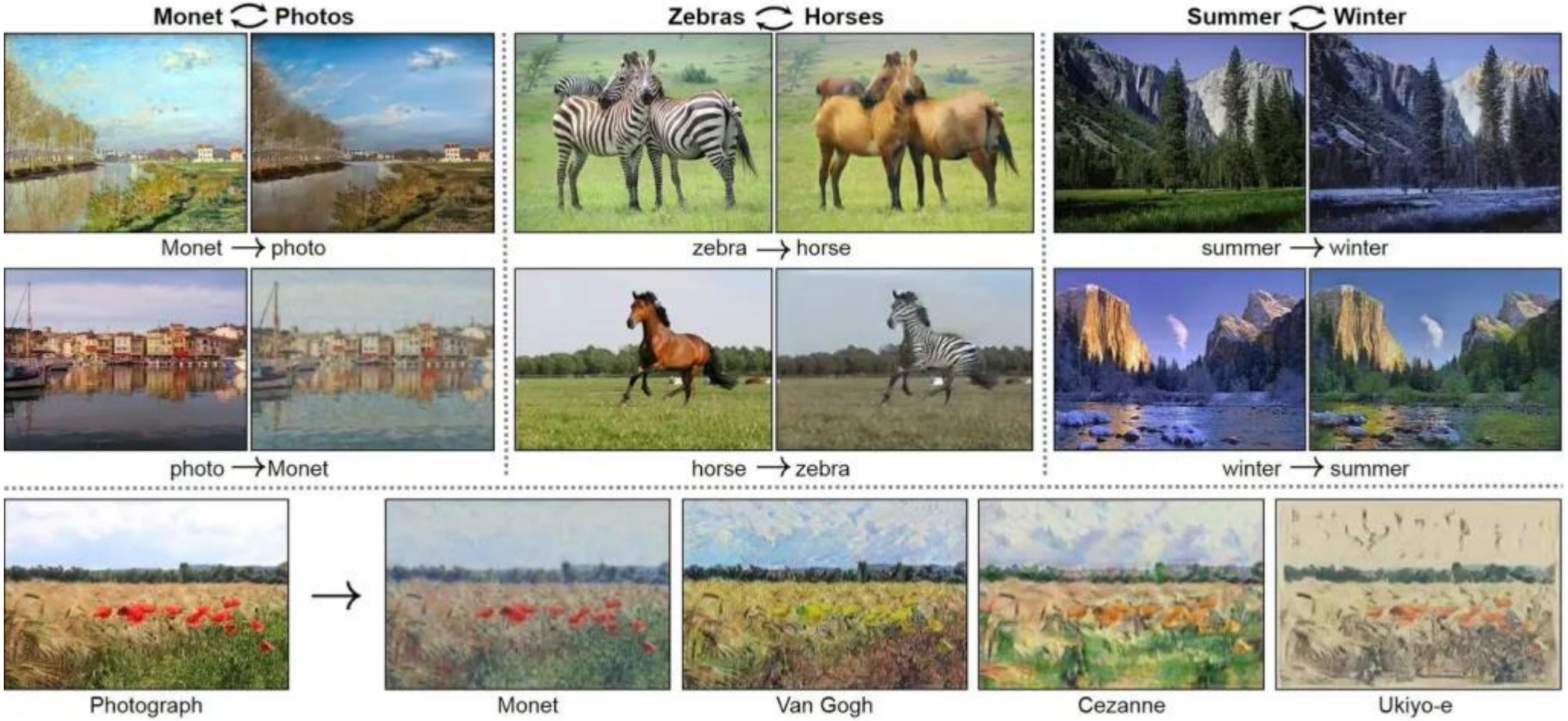
Application

Image-to-image translation

- Image-to-image translation is an application where a certain image B is transformed with the properties of A.



CycleGAN


Monet ⇄ Photos
Zebras ⇄ Horses
Summer ⇄ Winter
Monet → photo
zebra → horse
summer → winter
photo → Monet
horse → zebra
winter → summer
Photograph
Monet
Van Gogh
Cezanne
Ukiyo-e

Text-to-image synthesis

StackedGAN

- Stacked Generative Adversarial Networks (StackGAN) can generate images conditioned on text descriptions.

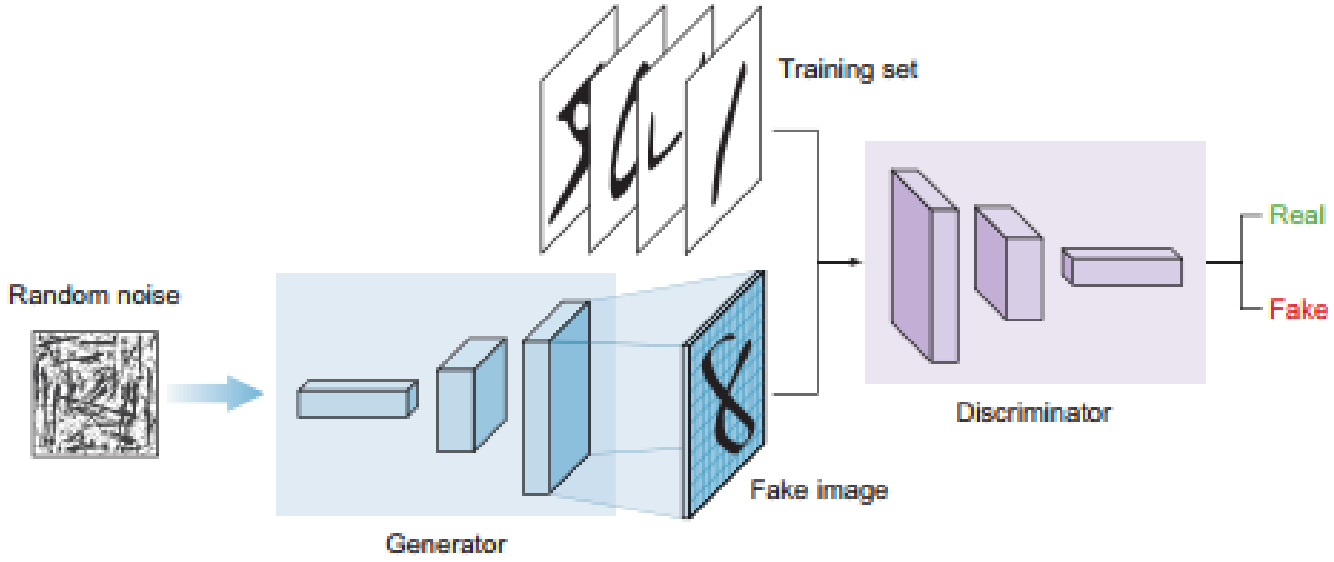
Face inpainting

- Facial inpainting, also known as face completion, is the task of generating plausible facial features for missing pixels in a face image.

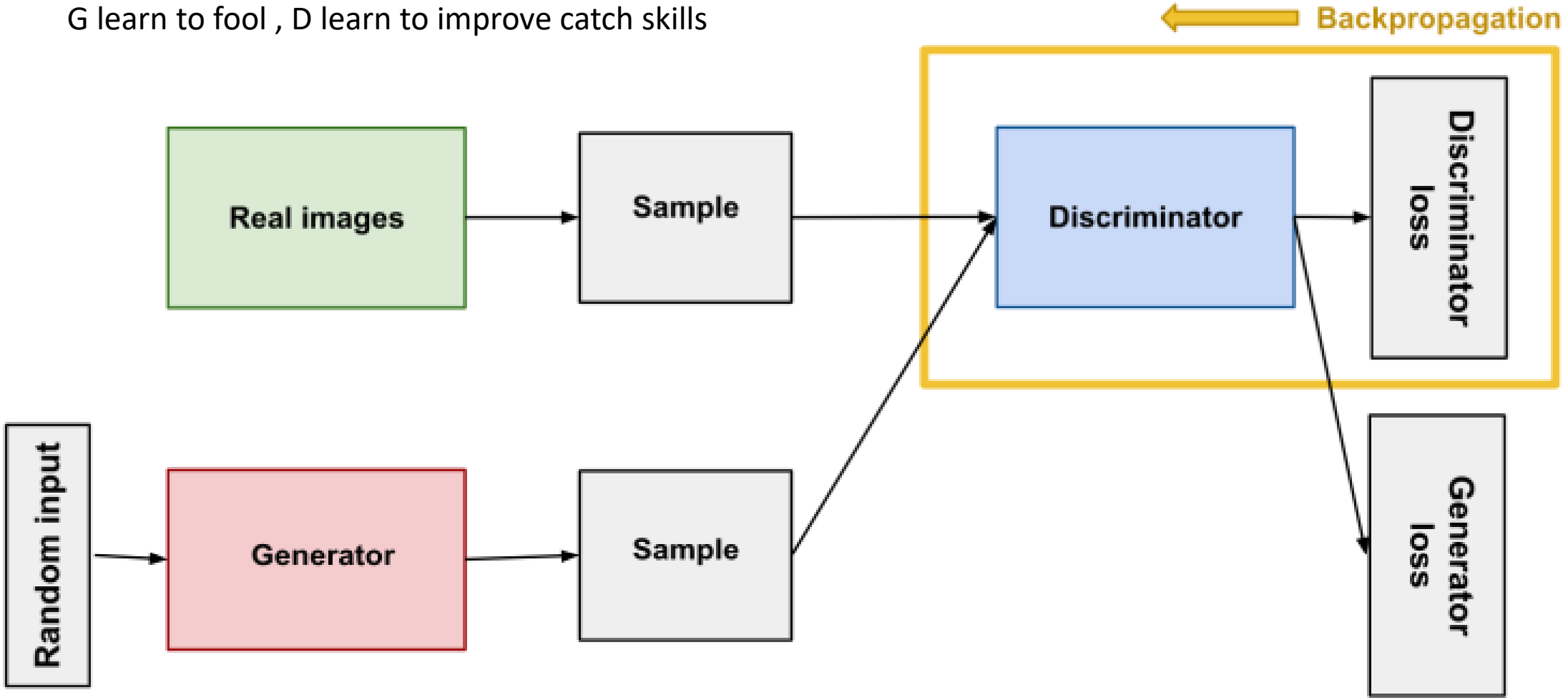
Principles of GANs

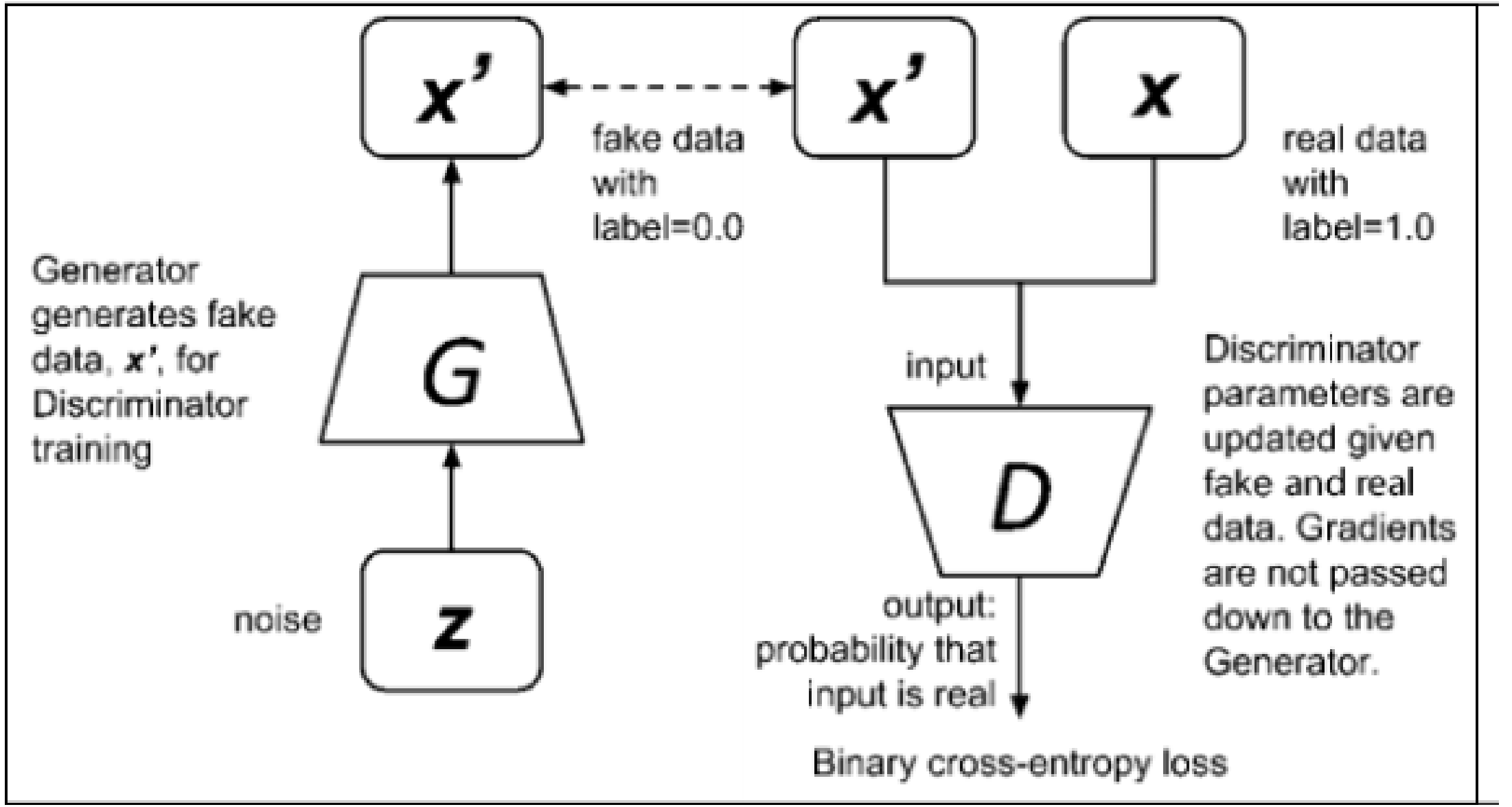
As you can see in the architecture diagram a GAN takes the following steps:

1. The generator takes in random numbers and returns an image.
2. This generated image is fed into the discriminator alongside a stream of images taken from the actual, ground-truth dataset.
3. The discriminator takes in both real and fake images and returns probabilities: numbers between 0 and 1, with 1 representing a prediction of authenticity and 0 representing a prediction of fake.



G learn to fool , D learn to improve catch skills




x'
fake data
with
label=0.0
x'
x
real data
with
label=1.0
Generator
generates fake
data, x', for
Discriminator
training
G
input
D
Discriminator
parameters are
updated given
fake and real
data. Gradients
are not passed
down to the
Generator.
noise
z
output:
probability that
input is real
Binary cross-entropy loss

Researchers can make computers generate content of the same or higher quality compared to that created by their human counterparts.
By learning to mimic any distribution of data, computers can be taught to create worlds that are similar to our own in any domain: images, music, speech, prose.
Ex: https://www.whichfaceisreal.com/index.php

https://generated.photos/face-generator/

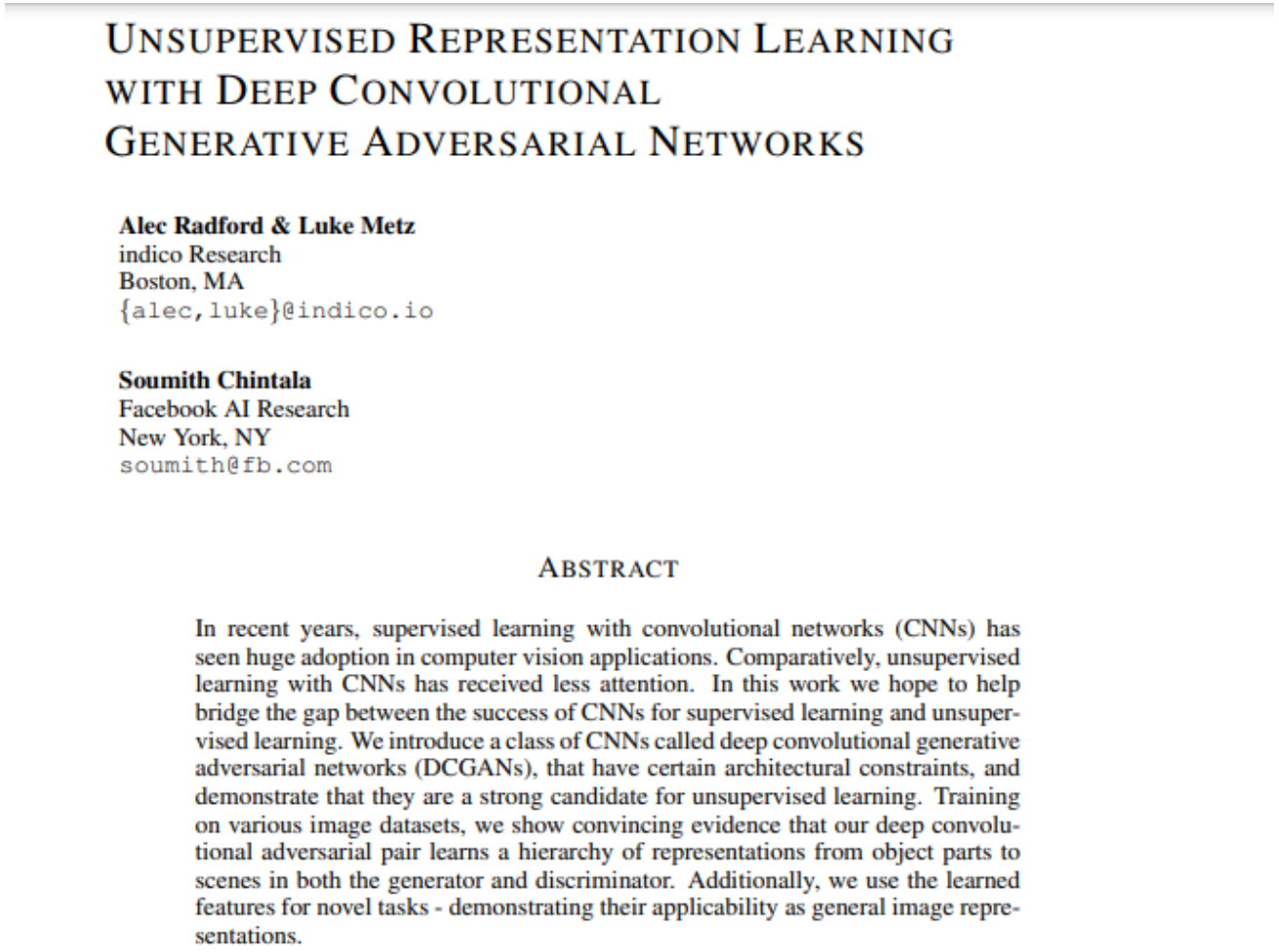
https://thispersondoesnotexist.com/

Deep convolutional GANs (DCGANs)

- In the original GAN paper in 2014, multi-layer perceptron (MLP) networks were used to build the generator and discriminator networks. However, since then, it has been proven that convolutional layers give greater predictive power to the discriminator, which in turn enhances the accuracy of the generator and the overall model. This type of GAN is called a deep convolutional GAN (DCGAN) and was developed by Alec Radford et al. in 2016

http://arxiv.org/abs/1511.06434

-



UNSUPERVISED REPRESENTATION LEARNING WITH DEEP CONVOLUTIONAL GENERATIVE ADVERSARIAL NETWORKS

Alec Radford & Luke Metz
indico Research
Boston, MA
{alec,luke}@indico.io

Soumith Chintala
Facebook AI Research
New York, NY
soumith@fb.com

ABSTRACT

In recent years, supervised learning with convolutional networks (CNNs) has seen huge adoption in computer vision applications. Comparatively, unsupervised learning with CNNs has received less attention. In this work we hope to help bridge the gap between the success of CNNs for supervised learning and unsupervised learning. We introduce a class of CNNs called deep convolutional generative adversarial networks (DCGANs), that have certain architectural constraints, and demonstrate that they are a strong candidate for unsupervised learning. Training on various image datasets, we show convincing evidence that our deep convolutional adversarial pair learns a hierarchy of representations from object parts to scenes in both the generator and discriminator. Additionally, we use the learned features for novel tasks - demonstrating their applicability as general image representations.

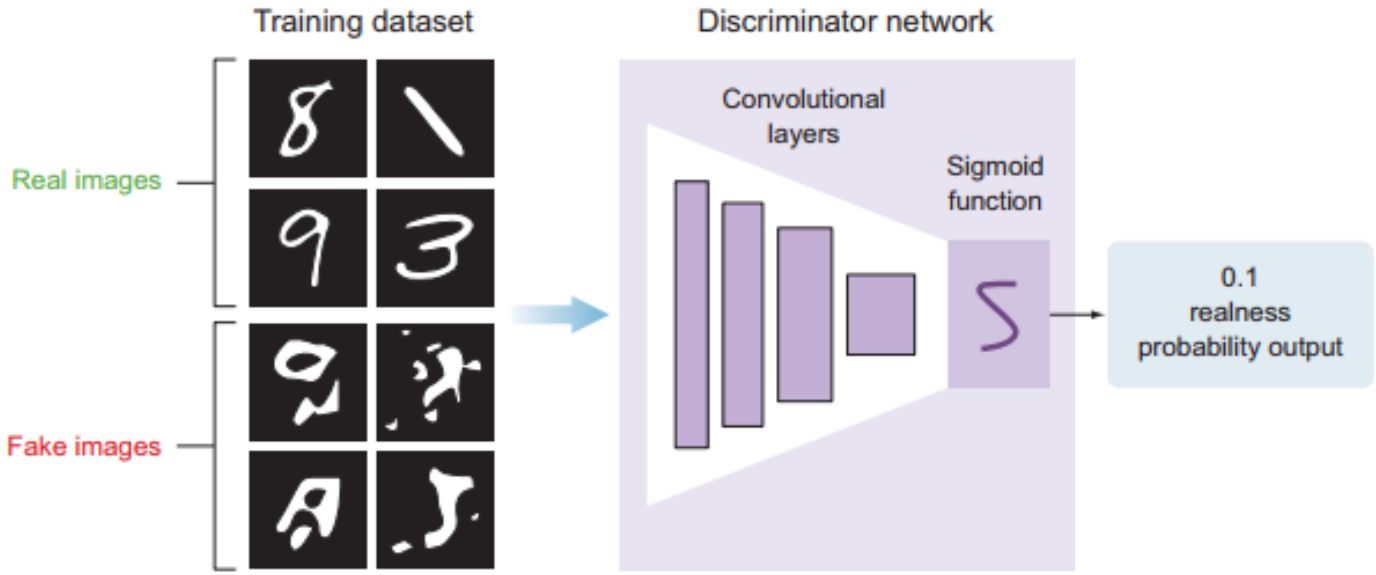
The discriminator model

The goal of the discriminator is to predict whether an image is real or fake. This is a typical supervised classification problem, so we can use the traditional classifier network that we learned about in the previous Lectures.

The network consists of stacked convolutional layers, followed by a dense output layer with a sigmoid activation function.

We use a sigmoid activation function because this is a binary classification problem: the goal of the network is to output prediction probabilities values that range between 0 and 1, where 0 means the image generated by the generator is fake and 1 means it is 100% real.

We feed the discriminator labeled images: fake (or generated) and real images. The real images come from the training dataset, and the fake images are the output of the generator model.



- We will first implement a discriminator_model function. In this code snippet, the shape of the image input is 28 × 28; you can change it as needed for your problem:

```
def discriminator_model():
    discriminator = Sequential()

    discriminator.add(Conv2D(32, kernel_size=3, strides=2,
                             input_shape=(28,28,1),padding="same"))

    discriminator.add(LeakyReLU(alpha=0.2))

    discriminator.add(Dropout(0.25))

    discriminator.add(Conv2D(64, kernel_size=3, strides=2, padding="same"))
    discriminator.add(ZeroPadding2D(padding=((0,1),(0,1))))

    discriminator.add(BatchNormalization(momentum=0.8))
    discriminator.add(LeakyReLU(alpha=0.2))
    discriminator.add(Dropout(0.25))

    discriminator.add(Conv2D(128, kernel_size=3, strides=2, padding="same"))
    discriminator.add(BatchNormalization(momentum=0.8))
    discriminator.add(LeakyReLU(alpha=0.2))
    discriminator.add(Dropout(0.25))
```

Instantiates a sequential model and names it discriminator

Adds a convolutional layer to the discriminator model

Adds a leaky ReLU activation function

Adds a dropout layer with a 25% dropout probability

Adds a second convolutional layer with zero padding

Adds a batch normalization layer for faster learning and higher accuracy

Adds a third convolutional layer with batch normalization, leaky ReLU, and a dropout

Adds the fourth convolutional layer with batch normalization, leaky ReLU, and a dropout

```
discriminator.add(Conv2D(256, kernel_size=3, strides=1, padding="same"))
discriminator.add(BatchNormalization(momentum=0.8))
discriminator.add(LeakyReLU(alpha=0.2))
discriminator.add(Dropout(0.25))

discriminator.add(Flatten())
discriminator.add(Dense(1, activation='sigmoid'))
```

Flattens the network and adds the output dense layer with sigmoid activation function

Prints the model summary

```
discriminator.summary()

img_shape = (28,28,1)
img = Input(shape=img_shape)

probability = discriminator(img)

return Model(img, probability)
```

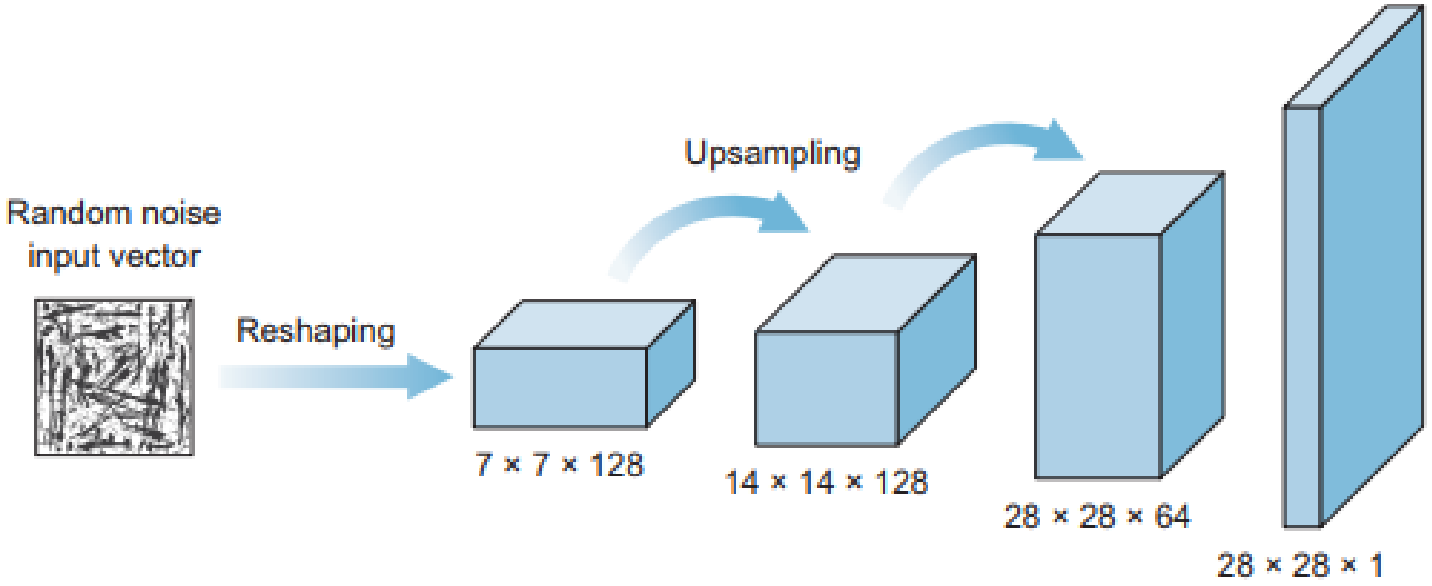
Sets the input image shape

Runs the discriminator model to get the output probability

Returns a model that takes the image as input and produces the probability output

The generator model

- The generator takes in some random data and tries to mimic the training dataset to generate fake images. Its goal is to trick the discriminator by trying to generate images that are perfect replicas of the training dataset. As it is trained, it gets better and better after each iteration. But the discriminator is being trained at the same time, so the generator has to keep improving as the discriminator learns its tricks.
- the generator model looks like an inverted ConvNet. The generator takes a vector input with some random noise data and reshapes it into a cube volume that has a width, height, and depth. This volume is meant to be treated as a feature map that will be fed to several convolutional layers that will create the final image.



UPSAMPLING TO SCALE FEATURE MAPS

- Traditional convolutional neural networks use pooling layers to downsample input images. In order to scale the feature maps, we use upsampling layers that scale the image dimensions by repeating each row and column of the input pixels.
- Keras has an upsampling layer (Upsampling2D) that scales the image dimensions by taking a scaling factor (size) as an argument:
  - **keras.layers.UpSampling2D(size=(2, 2))**
  - This line of code repeats every row and column of the image matrix two times, because the size of the scaling factor is set to (2, 2)

```
Input =  1, 2
         3, 4

Output = 1, 1, 2, 2
         1, 1, 2, 2
         3, 3, 4, 4
         3, 3, 4, 4
```

```
def generator_model():
    generator = Sequential()
    generator.add(Dense(128 * 7 * 7, activation="relu", input_dim=100))
    generator.add(Reshape((7, 7, 128)))
    generator.add(UpSampling2D(size=(2,2)))

    generator.add(Conv2D(128, kernel_size=3, padding="same"))
    generator.add(BatchNormalization(momentum=0.8))
    generator.add(Activation("relu"))
    generator.add(UpSampling2D(size=(2,2)))

# convolutional + batch normalization layers
    generator.add(Conv2D(64, kernel_size=3, padding="same"))
    generator.add(BatchNormalization(momentum=0.8))
    generator.add(Activation("relu"))

# convolutional layer with filters = 1
    generator.add(Conv2D(1, kernel_size=3, padding="same"))
    generator.add(Activation("tanh"))
    generator.summary()

    noise = Input(shape=(100,))
    fake_image = generator(noise)
    return Model(noise, fake_image)
```

Instantiates a sequential model and names it generator

Adds a dense layer that has a number of neurons = 128 × 7 × 7

Reshapes the image dimensions to 7 × 7 × 128

Upsampling layer to double the size of the image dimensions to 14 × 14

Adds a convolutional layer to run the convolutional process and batch normalization

Upsamples the image dimensions to 28 × 28

Prints the model summary

We don't add upsampling here because the image size of 28 × 28 is equal to the image size in the MNIST dataset. You can adjust this for your own problem.

Generates the input noise vector of length = 100. We use 100 here to create a simple network.

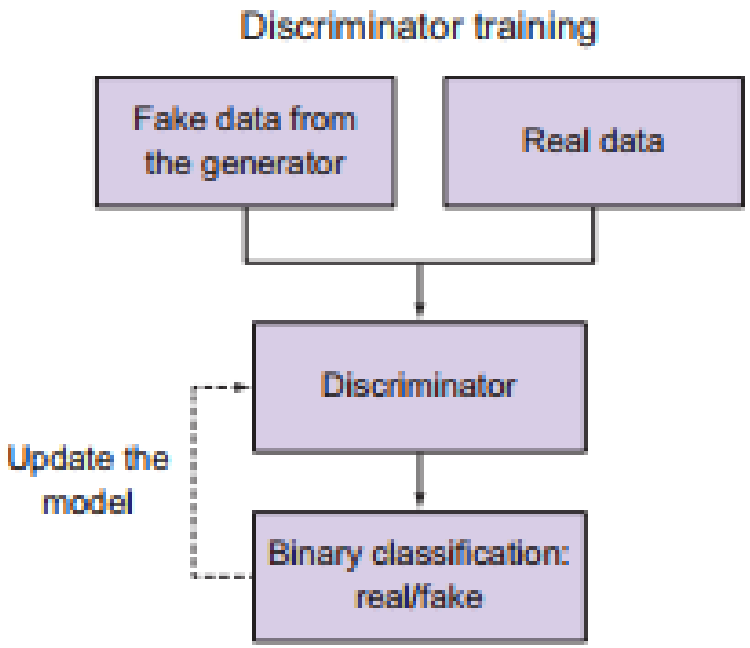
Runs the generator model to create the fake image

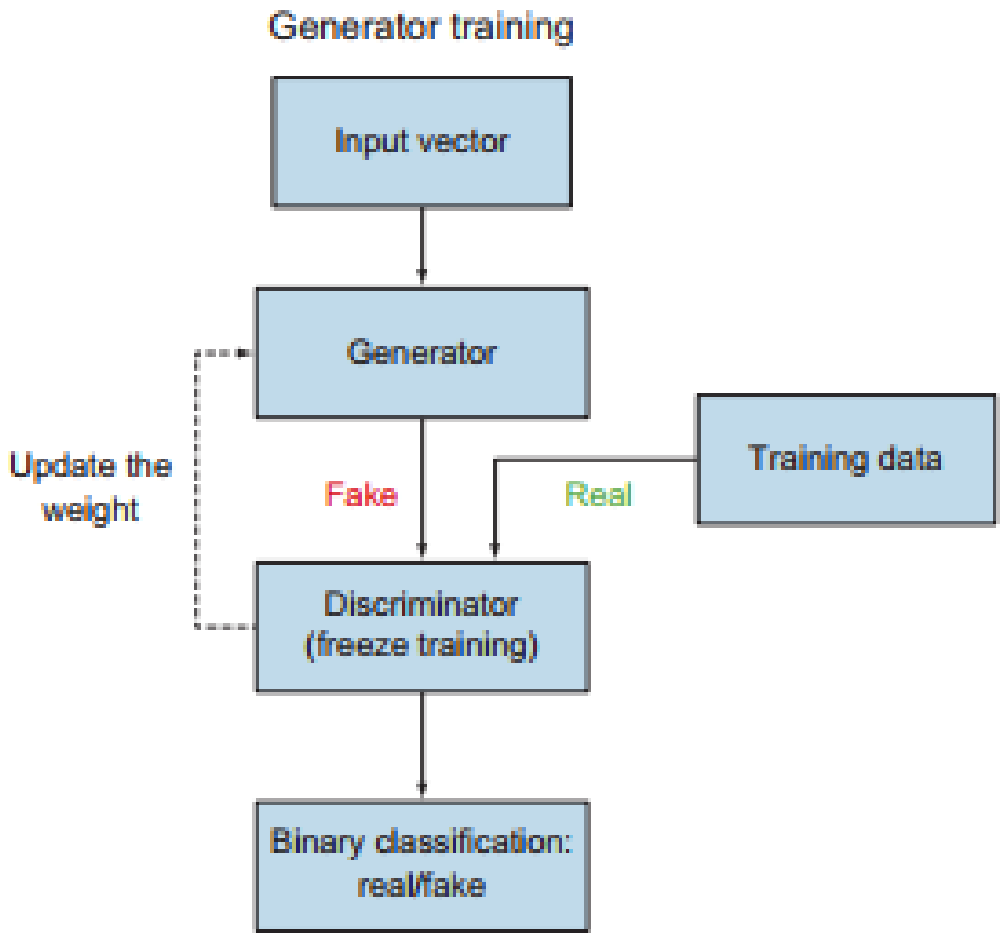
Returns a model that takes the noise vector as input and outputs the fake image

Training the GAN

- Now that we've learned the discriminator and generator models separately, let's put them together to train an end-to-end generative adversarial network.
- The discriminator is being trained to become a better classifier to maximize the probability of assigning the correct label to both training examples (real) and images generated by the generator (fake)
- The generator, on the other hand, is being trained to become a better forger, to maximize its chances of fooling the discriminator. Both networks are getting better at what they do.
  The process of training GAN models involves two processes:
- **1.** *Train the discriminator*. This is a straightforward supervised training process. The network is given labeled images coming from the generator (fake) and the training data (real), and it learns to classify between real and fake images with a sigmoid prediction output. Nothing new here.
- **2.** *Train the generator*. This process is a little tricky. The generator model cannot be trained alone like the discriminator. It needs the discriminator model to tell it whether it did a good job of faking images. So, we create a *combined network* to train the generator, composed of both discriminator and generator models.

Think of the training processes as two parallel lanes. One lane trains the discriminator alone, and the other lane is the combined model that trains the generator.





when training the combined model, we freeze the weights of the discriminator because this model focuses only on training the generator.

- Both processes follow the traditional neural network training process explained before. It starts with the feedforward process and then makes predictions and calculates and backpropagates the error. When training the discriminator, the error is backpropagated back to the discriminator model to update its weights; in the combined model, the error is backpropagated back to the generator to update its weights.

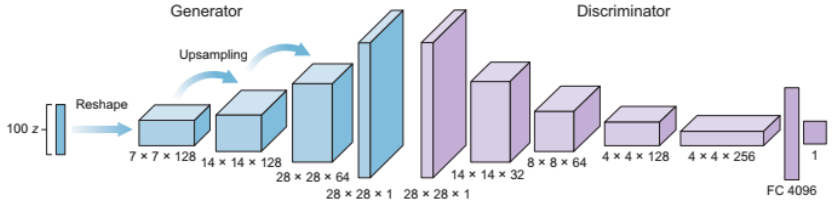
- TRAINING THE DISCRIMINATOR
  As we said before, this is a straightforward process. First, we build the model from the discriminator_model method that we created earlier in this chapter. Then we compile the model and use the **binary_crossentropy** loss function and an optimizer of your choice (we use Adam).

```
discriminator = discriminator_model()
discriminator.compile(loss='binary_crossentropy',optimizer='adam',
    metrics=['accuracy'])
```

- TRAINING THE GENERATOR (COMBINED MODEL)

- the generator needs the discriminator in order to be trained.
- When we want to train the generator, we freeze the weights of the discriminator model because the generator and discriminator have different loss functions pulling in different directions. If we don't freeze the discriminator weights, it will be pulled in the same direction the generator is learning so it will be more likely to predict generated images as real, which is not the desired outcome. Freezing the weights of the discriminator model doesn't affect the existing discriminator model that we compiled earlier when we were training the discriminator.

Think of it as having two discriminator models—this is not the case, but it is easier to imagine. Now, let's build the combined model:




```
generator = generator_model()

z = Input(shape=(100,))
image = generator(z)

discriminator.trainable = False

valid = discriminator(img)

combined = Model(z, valid)
```

Builds the generator

The generator takes noise as input and generates an image.

Freezes the weights of the discriminator model

The discriminator takes generated images as input and determines their validity.

The combined model (stacked generator and discriminator) trains the generator to fool the discriminator.

- Now that we have built the combined model, we can proceed with the training process as normal. We compile the combined model with a binary_crossentropy loss function and an Adam optimizer:

```
combined.compile(loss='binary_crossentropy', optimizer=optimizer)
g_loss = self.combined.train_on_batch(noise, valid)
```

Trains the generator (wants the discriminator to mistake images for being real)

Epoch: 0 | Epoch: 1,500 | Epoch: 2,500 | Epoch: 3,500 | Epoch: 5,500 | Epoch: 7,500 | Epoch: 9,500